

Database planning

The schema behind MinamiAI: four related tables covering accounts, conversations, the message transcript, and cross-session memory.

Overview

MinamiAI is an interactive AI companion: a chat window with real-time text-to-speech and a live, animated Live2D avatar. Four tables back the entire application, organized around a single principle — everything belongs to a user, and almost everything else hangs off that root.

Table	Backs
users	Accounts and roles
conversations	Chat sessions
messages	The transcript
memories	Cross-session recall

Relationships

Parent	Child	Cardinality	On delete
users	conversations	one to many	cascade
users	memories	one to many	cascade
conversations	messages	one to many	cascade
messages	memories	zero-or-one to many	set null

The one non-cascading relationship is `messages` to `memories`. Pruning old transcripts must not silently destroy the long-term memories derived from them, so `source_message_id` is nulled instead of cascading.

Tables

users

Account credentials and role, backing login and registration.

Column	Type	Notes
id	serial	primary key
username	varchar(32)	unique on <code>LOWER(username)</code>
password_hash	varchar(255)	bcrypt; never plaintext
role	varchar(16)	<code>user</code> or <code>admin</code> , check-constrained
created_at	timestamptz	

The `role` column gates the memory admin panel. Uniqueness is enforced on the lowercased username via a functional index, so `Izaya` and `izaya` cannot both be registered.

conversations

One chat session, sitting between users and messages so a session can be listed and titled independently, and so a transcript read is scoped to one session rather than a user's entire history.

Column	Type	Notes
id	serial	primary key
user_id	integer	→ users.id, cascade
title	varchar(120)	nullable
created_at	timestampz	

messages

The transcript, one row per turn. Read ordered by `created_at` ascending to rebuild the message array sent to Groq on every reply.

Column	Type	Notes
id	serial	primary key
conversation_id	integer	→ conversations.id, cascade
role	varchar(16)	<code>user</code> or <code>assistant</code>
content	text	non-empty after trim
created_at	timestampz	ordering key

memories

Long-term facts extracted from conversation in a background pass, owned by the user rather than the conversation so a fact learned in one session is available in the next — the mechanism behind Minami’s cross-session recall.

Column	Type	Notes
id	serial	primary key
user_id	integer	→ users.id, cascade
source_message_id	integer	→ messages.id, set null, nullable
content	text	the extracted fact
importance	smallint	1–5
expires_at	timestampz	null means permanent
created_at	timestampz	

The expiry invariant is enforced in the schema itself: tier 5 must have a null `expires_at`, every lower tier must have one set. A buggy extraction pass cannot write an immortal tier-1 memory about something trivial. `source_message_id` traces a memory back to the sentence that produced it, which is what the admin panel uses when Minami says something strange.

Indexes

Index	Serves
<code>users_username_lower_key</code>	case-insensitive login and registration uniqueness
<code>conversations_user_recent_idx</code>	recent-conversations sidebar
<code>messages_conversation_order_idx</code>	transcript rebuild on every single chat turn
<code>memories_user_importance_idx</code>	retrieving live memories for prompt injection
<code>memories_expiry_idx</code> (partial)	scheduled prune, skipping permanent rows
<code>memories_source_message_idx</code> (partial)	admin traceback, and the set-null sweep

`messages_conversation_order_idx` is the one that is not optional. Without it, every chat turn triggers a sequential scan that grows linearly with the user’s entire message history, so the app gets measurably slower the more someone uses it.

Security posture

- Passwords are bcrypt hashes. Nothing reversible is stored.
- Sessions are JWTs verified by middleware on every protected route.
- Every query touching `conversations`, `messages`, or `memories` is scoped by the user id taken from the verified token, never from a client-supplied parameter. This is the single most important rule in the codebase — without it, any user could read any other user's history by guessing an integer.
- All third-party API keys live server-side in environment variables. The browser talks only to the Express backend, which proxies Groq and ElevenLabs.
- Account deletion cascades to everything owned by that user. The correct privacy default, and irreversible, so the UI must confirm explicitly.

Known limitations

- **Transcript growth versus context window.** The transcript grows without bound; the model's context window does not. A windowed read plus a rolling summary is the eventual fix, and needs a `conversation_summaries` table that does not exist yet.
- **No semantic identity for memories.** Two rows saying essentially the same thing are, to the database, unrelated. A `pgvector` embedding column with a similarity check before insert is the long-term fix; a per-exchange cap on new memories plus admin cleanup is the interim mitigation.
- **Check constraints instead of enums.** `role` is a varchar with a `CHECK` constraint rather than a Postgres enum. Easier to migrate, weaker at the schema level.
- **No audio cache.** Every reply is synthesized from scratch even when the same text is produced twice. A cache table keyed on a hash of (text, voice settings) is the obvious addition if TTS cost becomes a real constraint.
- **Serial rather than UUID primary keys.** Fine behind proper authorization scoping, but they leak rough record counts. Worth revisiting only if a resource ever becomes shareable by URL.

Applying the migration

Fresh database:

```
createdb minamiai
psql "$DATABASE_URL" -f migrations/001_init.sql
```

The script is idempotent at the object level, so re-running it is safe.